

SKALA

KDT DevOps

DevOps 이해 및 활용

2026.03

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.



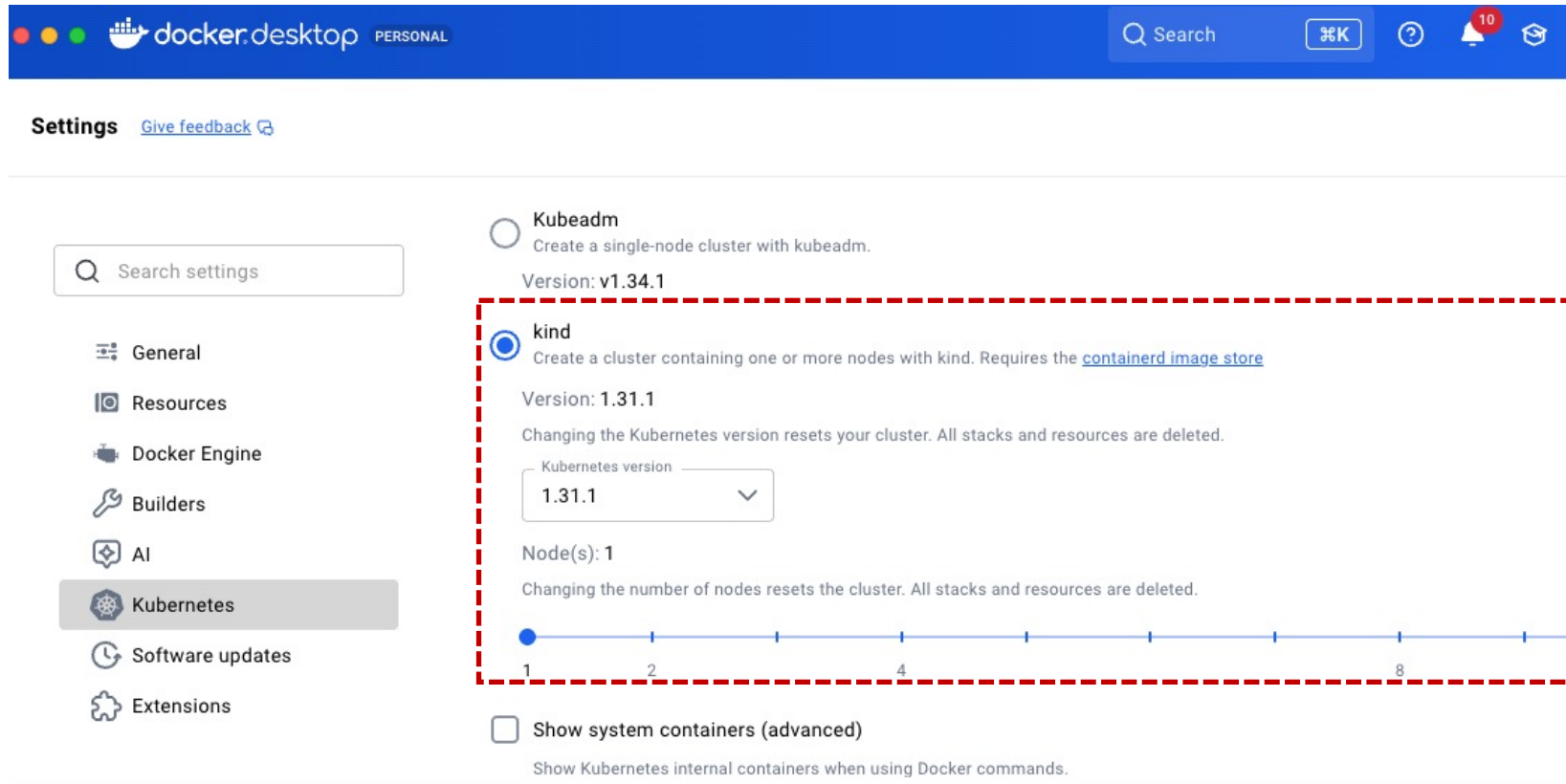
- 1. DevOps 개요**
- 2. Git 이해 및 활용**
- 3. Jenkins**
- 4. Tekton**
- 5. Github Action**
- 6. ArgoCD**

0. 로컬 PC 내 쿠버네티스 환경 세팅

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

도커 내 쿠버네티스 기능 활성화 (Kind 선택)

0. 쿠버네티스 환경 세팅



🚀 이미 K8s 과정을 마친 경우 AWS EKS 클러스터 대상으로만 세팅하면 됨.

kubectl 활용

kubectl 설치

\$ brew install kubectl

#.zshrc 내 하기 내용 반영 (앞으로는 "k" 만 사용한다)

alias k=kubectl

반영한 alias 적용

\$ source .zshrc

다중 쿠버네티스 클러스터 접근 시 활용되는 명령어

\$ k config get-contexts

\$ k config current-context

\$ k config rename-context <old-name> <new-name>

\$ k config use-context <context-name>

docker-desktop 클러스터 확인

✅ skala-2025 클러스터는 AWS EKS 클러스터 (k8s 시간에 활용 예정)

```
{steve} k config get-contexts
```

CURRENT	NAME	CLUSTER
*	kind	docker-desktop
	skala-2025	arn:aws:eks:ap-northeast-2:881490135253:cluster/skala-2025

🚀 이미 K8s 과정을 마치신 경우 AWS EKS 클러스터 대상으로만 세팅하면 됨.

1. DevOps 개요

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

클라우드 네이티브?

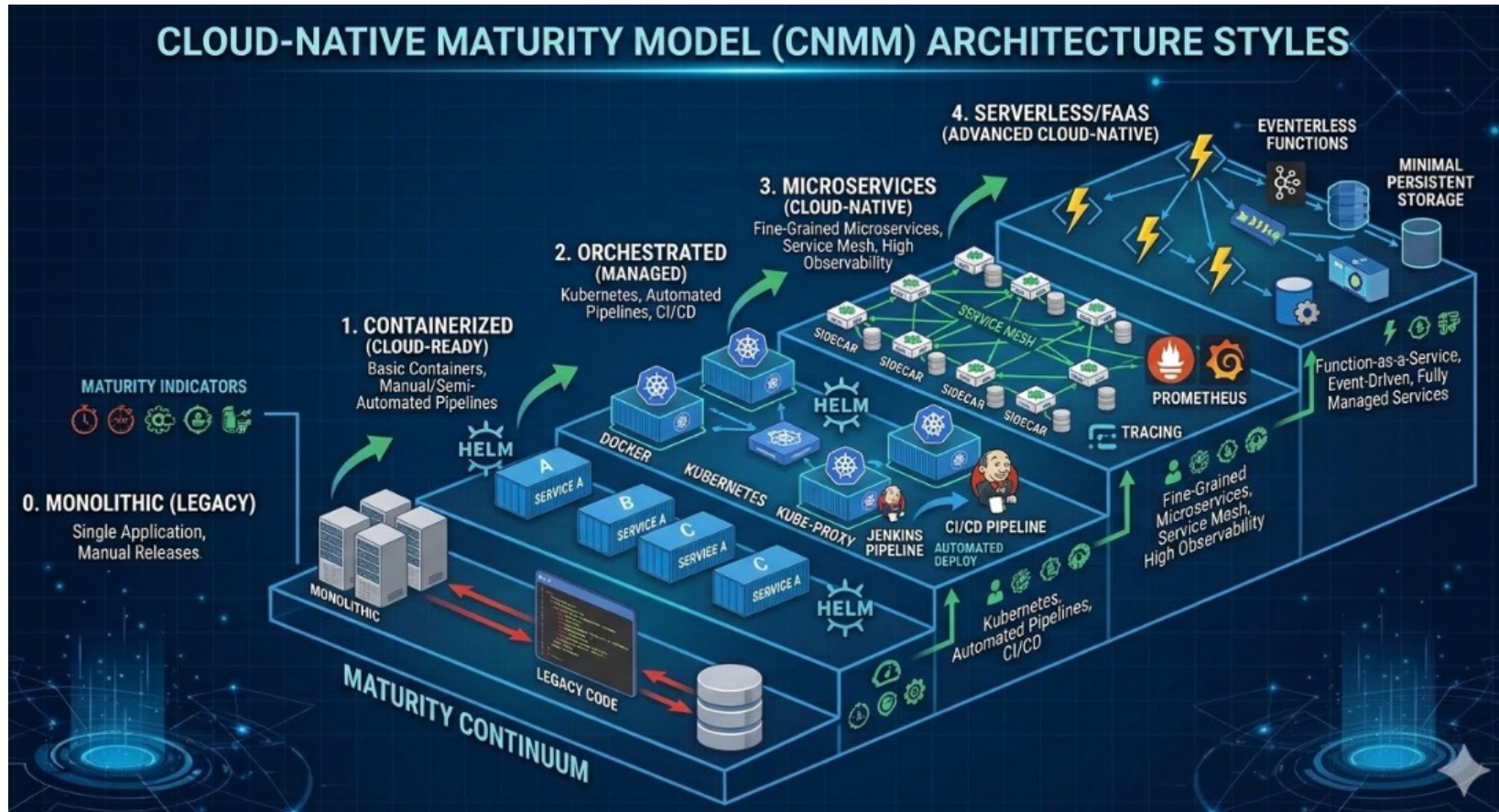
- ✅ 애플리케이션을 클라우드 환경에 가장 적합한 방식으로 설계, 개발, 배포, 운영하는 접근 방식.
- ✅ 클라우드 네이티브의 주요 특징인 확장성, 탄력성, 복원력, 자동화, 민첩성을 최대한 적용

클라우드 네이티브의 5대 핵심요소



클라우드 네이티브?

항목	전통적 아키텍처	클라우드 네이티브
배포 방식	수동 배포, 긴 릴리즈 주기	자동화된 CI/CD, 수시 배포
스케일링	수직 스케일링 (스펙 업)	수평 스케일링 (인스턴스 추가)
장애 처리	단일 장애점, 수동 복구	자동 복구, 회로 차단기 패턴
인프라 관리	물리 서버, 수동 설정	IaC, 선언적 API로 자동화
개발 방식	모놀리식, 폭포수 개발	마이크로서비스, 애자일/DevOps
확장성	제한적, 비용 높음	무제한, 비용 최적화 가능
데이터 관리	중앙 집중식 DB	서비스별 독립 DB, 분산 캐시



DevOps?

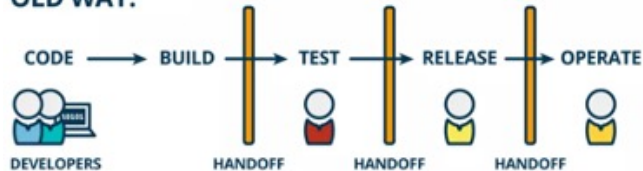
개발 과 운영을 통합해서 협업을 극대화하기 위한 문화와 철학

Dev ←-----→ Ops

책임 전가 / 책임 회피
 난 운영 몰라 / 난 개발 안했는데,
 던지면 그만 / 쓰레기 던지지마..
 신기술 도입해야지 / 검증 안된 기술 도입 안돼

weeks/months/years

OLD WAY:



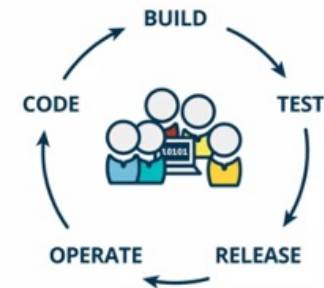
Cyclic delivery
of earlier times

Dev + Ops

협업/협동을 통한 문제 해결
 내가 끝까지 책임져야지 (Full Stack 인력)
 처음부터 쓰레기를 만들지 말자..
 신기술 도입해서 검증까지 해보자 (Fast Failed)

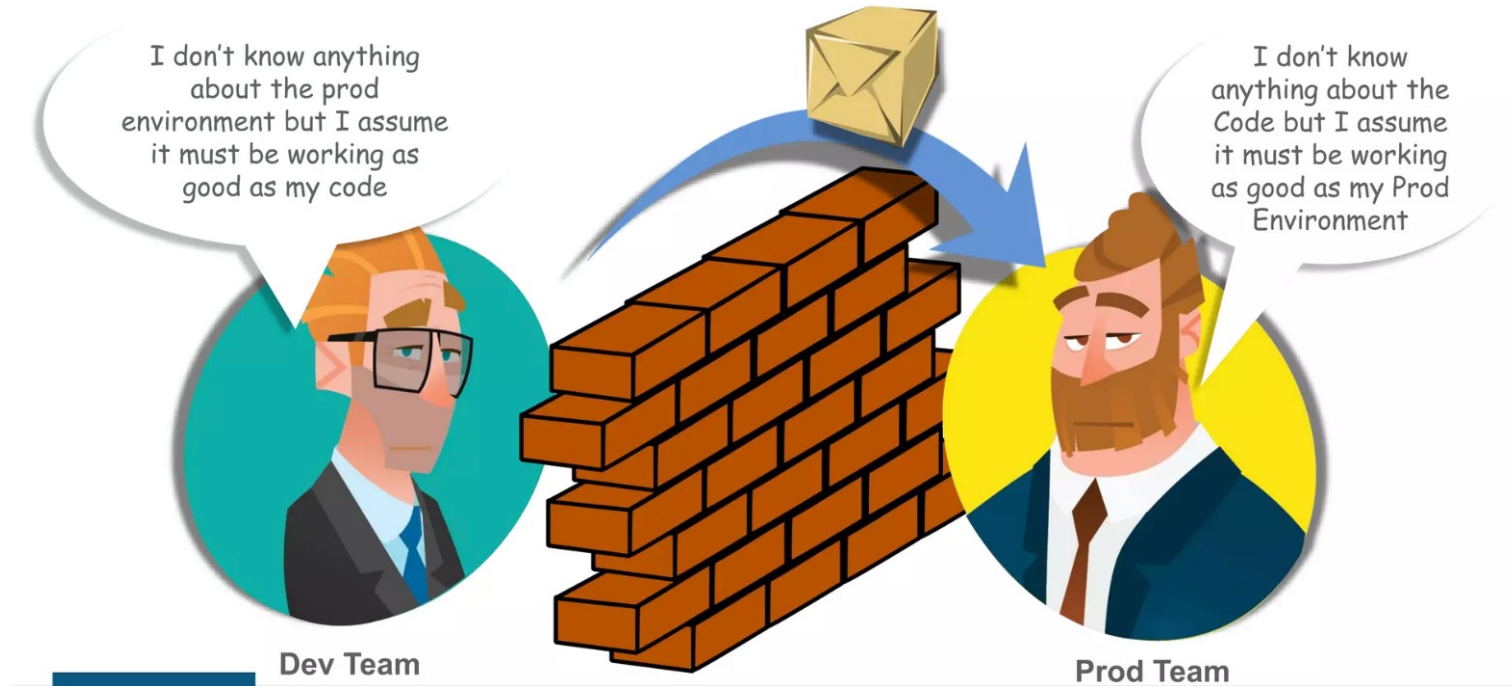
hours/days!

NEW WAY:



continuous delivery
of modern times

개발때는 잘 동작하는데 오픈 시 항상 장애가 ...

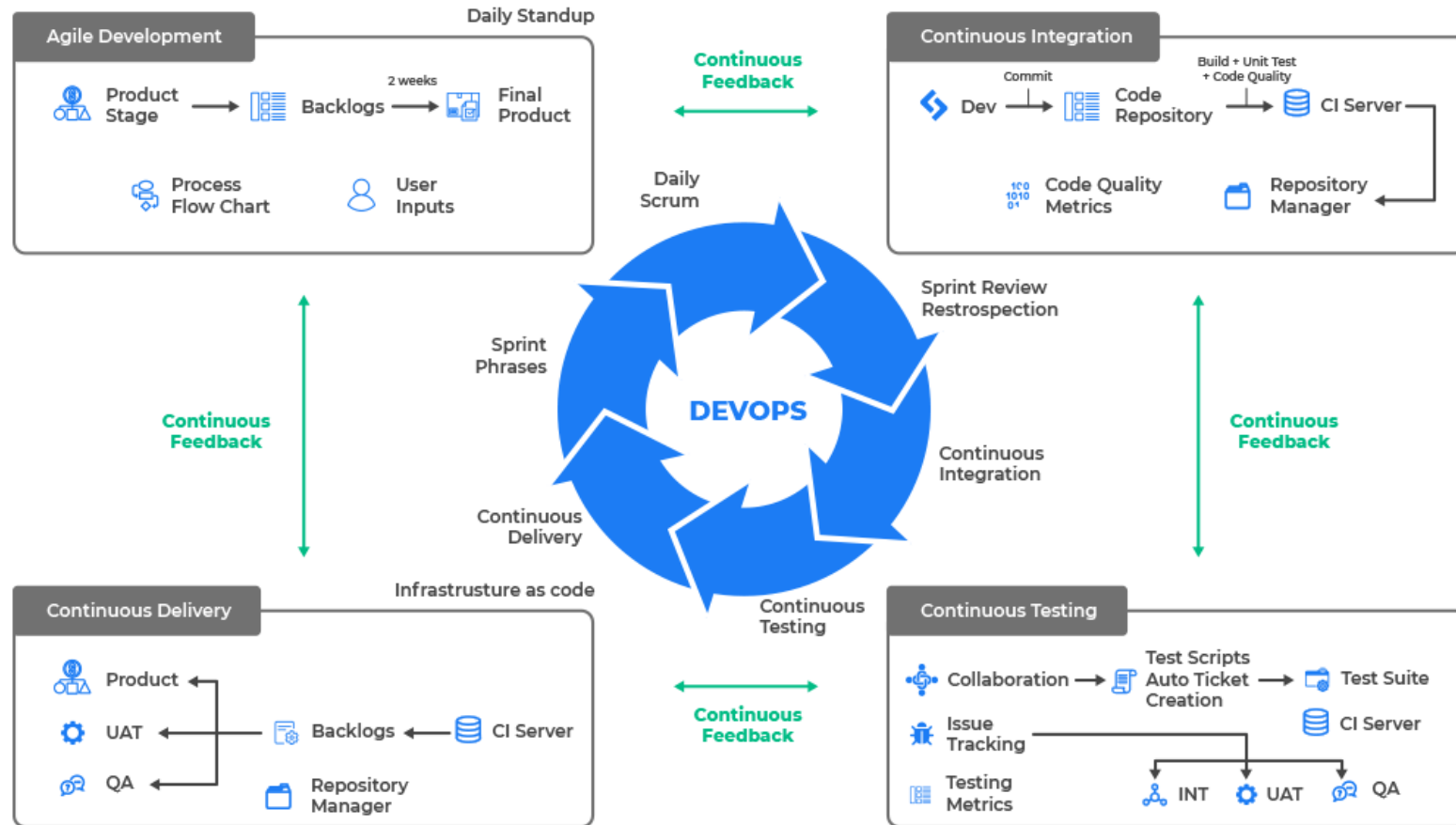


DevOps?



DevOps?

1. DevOps 개요



DevOps 도입 기업은 더 많은 작업을 간단하게 처리하며, 최고의 속도, 기능 및 혁신으로 제품을 제공

DevOps의 Business Benefits

애플리케이션의 배포 빈도와 속도를 개선하여
계획단계부터 안정적 최종 배포까지 시간 단축 -
비즈니스와 기술의 변화에 신속 대응



DevOps Practice

DevOps 성과가 우수한 기업은 코드를 더 빠르게 배포하고,
문제를 더 적게 발생시키며, 인시던트를 더 빠르게 해결. -
DORA's 2019 State of DevOps Report



208x
MORE
frequent code
deployments

106x
FASTER
lead time from
commit to deploy



2,604x
FASTER
time to recover
from incidents

7x
LOWER
change failure rate
(changes are 1/7 as likely to fail)



DevOps?

DevOps는 소프트웨어 개발(Development)과 IT 운영(Operations)의 합성어로, 두 분야 간의 협업과 통합을 강조하는 문화 및 실천 방법론이다. DevOps는 소프트웨어 개발과 운영 팀 간의 장벽을 허물고, 소프트웨어 제품의 개발, 테스트, 배포, 운영을 자동화하고 최적화하여 더 빠르고 안정적인 소프트웨어 릴리스를 목표로 한다.

✅ DevOps 주요 개념

✓ 문화(Culture):

- DevOps는 협업과 커뮤니케이션을 중시하는 문화이다.
- 개발자와 운영자가 함께 일하며, 공동의 목표를 설정하고 이를 달성하기 위해 협력한다.
- 지속적인 피드백과 개선을 통해 팀의 효율성을 높인다.

✓ 자동화(Automation):

- 빌드, 테스트, 배포, 모니터링 등 소프트웨어 개발 및 운영의 여러 단계를 자동화하여 인적 오류를 줄이고 효율성을 높인다.
- CI/CD(지속적 통합 및 지속적 배포) 파이프라인을 통해 코드 변경 사항을 자동으로 테스트하고 배포한다.

DevOps?

✓ 통합(Integration):

다양한 도구와 시스템을 통합하여 개발과 운영의 경계를 허물고, 데이터와 프로세스를 일관되게 관리한다.
예를 들어, 코드 리포지토리, 빌드 서버, 테스트 도구, 배포 플랫폼 등을 하나의 파이프라인으로 연결한다.

✓ 모니터링 및 로깅(Monitoring and Logging):

애플리케이션과 인프라의 상태를 지속적으로 모니터링하여 문제를 조기에 발견하고 해결한다.
로그 데이터를 분석하여 성능을 최적화하고, 사용자 경험을 개선한다.

✓ 협업 및 커뮤니케이션(Collaboration and Communication):

DevOps는 팀 간의 원활한 커뮤니케이션을 촉진하여 문제 해결 속도를 높이고, 혁신을 가속화한다.
협업 도구와 플랫폼을 활용하여 실시간으로 정보를 공유하고, 피드백을 주고받는다.

DevOps?

✓ DevOps의 이점

- ✓ **빠른 배포 속도:** 자동화된 파이프라인을 통해 소프트웨어를 더 빠르게 배포할 수 있다.
- ✓ **높은 품질:** 지속적인 테스트와 모니터링을 통해 소프트웨어의 품질을 유지하고 개선한다.
- ✓ **효율성 향상:** 반복적이고 수동적인 작업을 자동화하여 팀의 생산성을 높인다.
- ✓ **리스크 감소:** 작은 단위의 변경 사항을 자주 배포함으로써 리스크를 줄이고, 문제 발생 시 빠르게 대응할 수 있다.
- ✓ **고객 만족도 향상:** 더 빠르고 안정적인 소프트웨어 릴리스를 통해 고객의 요구에 신속하게 대응할 수 있다.

DevOps를 실현하는 방법 중 하나인 CI/CD Pipeline

CI/CD 파이프라인은 소프트웨어 개발에서 코드 변경 사항을 자동으로 빌드, 테스트, 배포하는 일련의 프로세스를 의미한다.

CI/CD는 지속적 통합(Continuous Integration)과 지속적 배포(Continuous Deployment) 또는 지속적 전달(Continuous Delivery)을 포함하며, 소프트웨어 개발의 효율성과 품질을 높이는 데 중점을 둔다.

✓ CI/CD 파이프라인의 주요 구성 요소

지속적 통합(Continuous Integration, CI)

개념: 개발자들이 코드 변경 사항을 중앙 리포지토리에 자주 병합하는 프로세스이다.

목적: 코드 변경 사항을 자동으로 빌드하고 테스트하여 통합 시 발생할 수 있는 문제를 조기에 발견하고 해결한다.

도구: Jenkins, GitLab CI, Travis CI, CircleCI 등.

지속적 전달(Continuous Delivery, CD)

개념: CI의 결과물을 자동으로 준비하여 프로덕션 환경에 배포할 수 있는 상태로 만드는 프로세스이다.

목적: 코드가 언제든지 프로덕션에 배포될 수 있도록 준비 상태를 유지한다.

특징: 배포는 여전히 수동으로 수행될 수 있으며, 승인 절차가 포함될 수 있다.

지속적 배포(Continuous Deployment, CD)

개념: 지속적 전달의 확장으로, 코드 변경 사항이 자동으로 프로덕션 환경에 배포되는 프로세스이다.

목적: 모든 코드 변경 사항이 자동으로 배포되어 사용자에게 즉시 제공된다.

특징: 완전한 자동화로, 수동 개입 없이 배포가 이루어진다.

CI/CD Pipeline

✓ CI/CD 파이프라인의 단계

1. 코드 커밋

개발자가 코드 변경 사항을 버전 관리 시스템(예: Git)에 커밋한다.

2. 빌드

코드가 자동으로 빌드되어 실행 가능한 애플리케이션 패키지로 변환된다.
종속성 설치 및 환경 설정이 포함될 수 있다.

3. 테스트

자동화된 테스트가 실행되어 코드의 기능과 품질을 검증한다.
유닛 테스트, 통합 테스트, 기능 테스트 등이 포함된다.

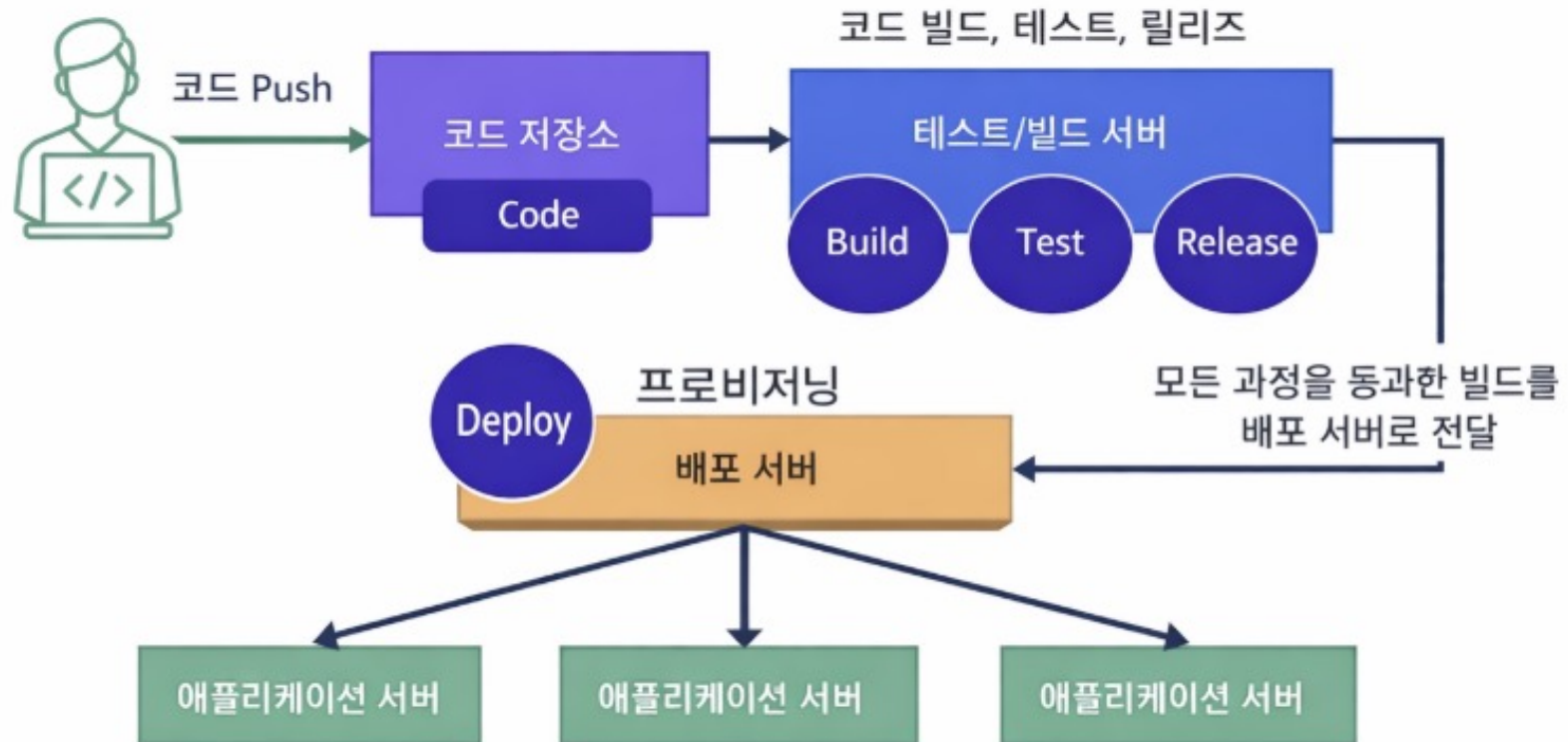
4. 배포

테스트를 통과한 코드가 Staging 또는 Production 환경에 배포된다.
지속적 배포의 경우, 이 단계가 자동으로 수행된다.

5. 모니터링 및 피드백

배포된 애플리케이션의 성능과 안정성을 모니터링한다.
사용자 피드백과 모니터링 데이터를 통해 지속적인 개선을 도모한다.

CI/CD Pipeline



2. Git 이해 및 활용

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Git 이해

Git은 분산 버전 관리 시스템(DVCS)으로, 소프트웨어 개발 프로젝트에서 코드의 변경 이력을 관리하고 협업을 용이하게 하는 도구이다

버전 관리(Version Control)

Git은 코드의 변경 이력을 추적하여 언제든지 이전 버전으로 되돌릴 수 있게 한다.

각 변경 사항은 커밋(commit)으로 저장되며, 커밋에는 변경된 파일과 변경 내용에 대한 설명이 포함된다.

분산 시스템(Distributed System)

Git은 중앙 서버 없이도 각 개발자가 로컬 저장소를 가지고 작업할 수 있는 분산 시스템이다.

모든 개발자는 전체 코드베이스의 복사본을 로컬에 저장하며, 네트워크 연결 없이도 작업이 가능하다.

브랜치(Branch)

브랜치는 독립적인 작업 흐름을 가능하게 하며, 새로운 기능 개발이나 버그 수정을 위한 별도의 작업 공간을 제공한다.

브랜치를 사용하면 메인 코드베이스에 영향을 주지 않고도 다양한 시도를 할 수 있다.

병합(Merge)

브랜치에서 작업한 내용을 메인 브랜치에 통합하는 과정이다.

병합 시 충돌(conflict)이 발생할 수 있으며, 이를 수동으로 해결해야 한다.

리포지토리(Repository)

코드와 그 변경 이력을 저장하는 저장소이다.

로컬 리포지토리와 원격 리포지토리로 나뉘며, 원격 리포지토리는 GitHub, GitLab, Bitbucket과 같은 플랫폼에 호스팅될 수 있다.

Git 이해

✓ Git의 주요 명령어

git init: 새로운 Git 리포지토리를 초기화한다.

ex) git init

git init my-project (새 디렉토리 생성 후 초기화)

cd my-project

git clone: 원격 리포지토리를 복제하여 로컬에 저장한다.

ex) git clone https://github.com/username/repository.git

git clone https://github.com/username/repository.git my-folder (특정 디렉토리로 복제)

git add: 변경된 파일을 스테이징 영역에 추가한다.

ex) git add filename.txt (filename.txt 파일을 스테이징 영역에 추가)

git add . (모든 변경된 파일을 스테이징 영역에 추가)

git commit: 스테이징 영역의 변경 사항을 커밋으로 저장한다.

ex) git commit -m "Add new feature"

git push: 로컬 커밋을 원격 리포지토리에 업로드한다.

ex) git push origin main (로컬의 main 브랜치를 원격 리포지토리의 main 브랜치에 업로드한다.)

git pull: 원격 리포지토리의 변경 사항을 로컬로 가져온다.

ex) git pull origin main (원격 리포지토리의 main 브랜치에서 변경 사항을 가져와 로컬 main 브랜치에 병합한다)

Git 이해

✓ Git의 주요 명령어

원격 저장소 연결

원격 저장소 추가

git remote add origin https://github.com/username/repository.git

원격 저장소 목록

git remote -v

원격 저장소 URL 변경

git remote set-url origin https://github.com/username/new-repo.git

원격 저장소 제거

git remote remove origin

Git 이해

✓ Git의 주요 명령어

git branch: 브랜치를 생성하거나 나열한다.

ex) git branch
git branch new-feature

git checkout: 특정 브랜치나 커밋으로 전환한다.

ex) git checkout new-feature (new-feature 브랜치로 전환)
git checkout -b another-feature (another-feature라는 새로운 브랜치를 생성하고 전환)

git merge: 다른 브랜치를 현재 브랜치에 병합한다.

ex) git merge new-feature (new-feature 브랜치를 현재 브랜치에 병합)

git status: 현재 작업 디렉토리의 상태를 확인한다.

ex) git status (변경된 파일, 스테이징 상태 등을 확인)

git log: 커밋 이력을 확인한다.

ex) git log (커밋 히스토리를 나열)

전역 설정 (모든 프로젝트에 적용)

```
git config --global user.name "홍길동"
```

```
git config --global user.email "hong@example.com"
```

현재 프로젝트에만 적용

```
git config user.name "홍길동"
```

```
git config user.email "hong@example.com"
```

설정 확인

```
git config --list
```

```
git config user.name
```

```
git config user.email
```

기본 브랜치 이름 설정

```
git config --global init.defaultBranch main
```

**.gitignore는 Git이 추적하지 않도록 제외할 파일/폴더 목록을 적는
설정 파일 (.gitignore 파일 내 ".env" 반영)**

버전에 파일 담기

```
git add filename.py
```

모든 파일 담기

```
git add .
```

git add 취소하기

```
git rm --cached <file>
```

버전으로 묶어주기

```
git commit
```

커밋 메시지 한 번에 작성

```
git commit -m "comment"
```

커밋 이력 확인

```
git log
```

파일 변경 사항 자세히 보기

```
git diff
```

로컬에서 Github Repo. 연결

2. Git 이해 및 활용

 **git-test** Private

**Set up GitHub Copilot**
Use GitHub's AI pair programmer to autocomplete suggestions as you code.
[Get started with GitHub Copilot](#)

Quick setup — if you've done this kind of thing before
[Set up in Desktop](#) or [HTTPS](#) [SSH](#) <https://github.com/cc202501/git-test.git>
Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [CONTRIBUTING](#) file.

...or create a new repository on the command line

```
echo "# git-test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/cc202501/git-test.git
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/cc202501/git-test.git
git branch -M main
git push -u origin main
```

 **Add**
[Search](#)
[Invite](#)

```
stevejung@Stevesui-iMac ~/KDT/Sample/git-test
stevejung@Stevesui-iMac ~/KDT/Sample/git-test
/Users/stevejung/KDT/Sample/git-test/.git/ 안의 빈 깃 저장소를 다시 초기화했습니다
stevejung@Stevesui-iMac ~/KDT/Sample/git-test $ main $ ls
README.md
stevejung@Stevesui-iMac ~/KDT/Sample/git-test $ main $ git add .
stevejung@Stevesui-iMac ~/KDT/Sample/git-test $ main + $ git commit -m "first commit"
[main (최상위-커밋) 8bfbc55] first commit
1 file changed, 1 insertion(+)
create mode 100644 README.md
stevejung@Stevesui-iMac ~/KDT/Sample/git-test $ main $ git branch -M main
stevejung@Stevesui-iMac ~/KDT/Sample/git-test $ main $ git remote add origin https://github.com/cc202501/git-test.git
stevejung@Stevesui-iMac ~/KDT/Sample/git-test $ main $ git push -u origin main
오브젝트 나열하는 중: 3, 완료.
오브젝트 개수 세는 중: 100% (3/3), 완료.
오브젝트 쓰는 중: 100% (3/3), 230 bytes | 230.00 KiB/s, 완료.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cc202501/git-test.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

 **git-test** Private [Unwatch](#) 1

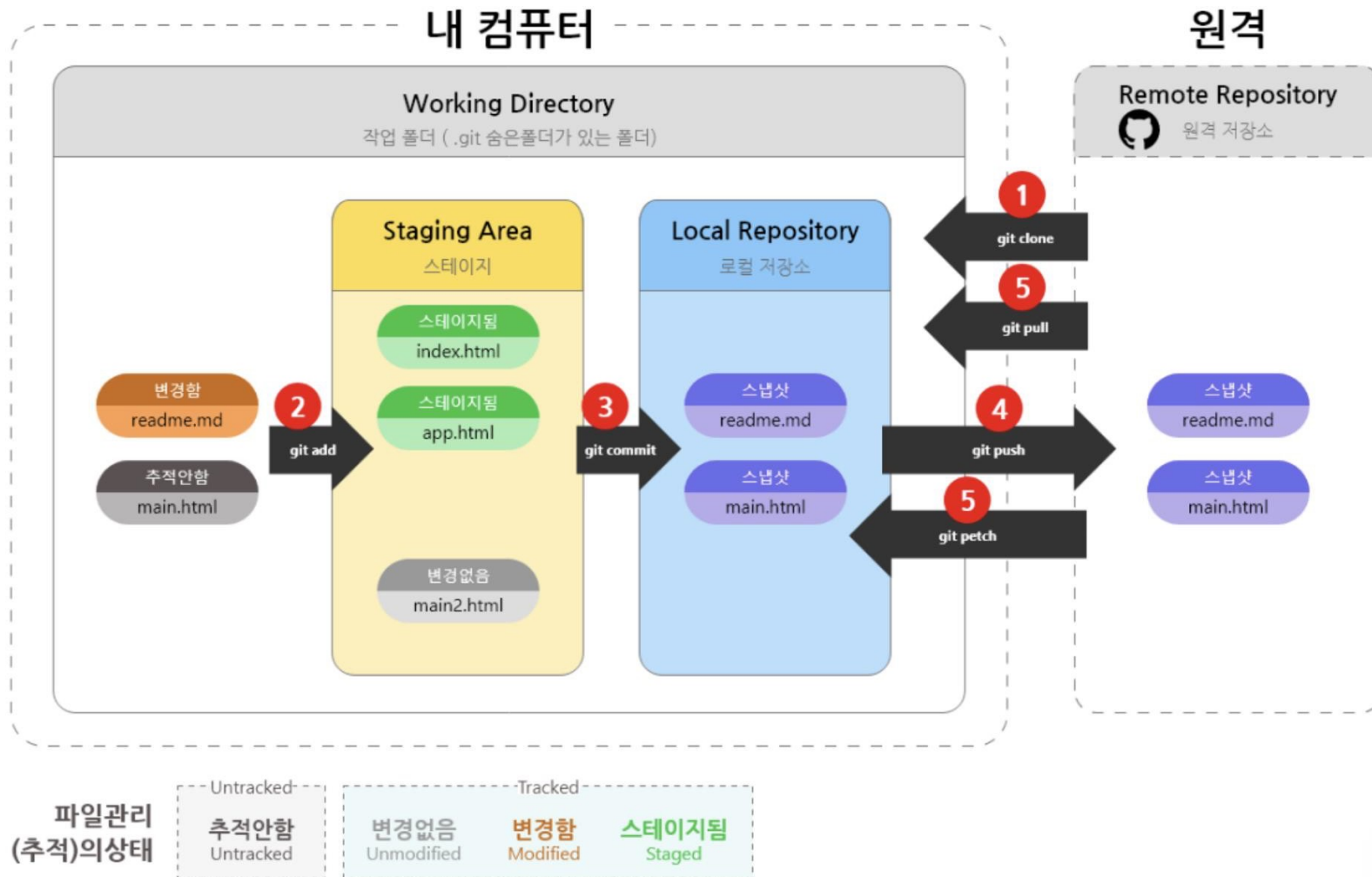
[main](#)   [+](#) [Code](#)

 **cc202501** first commit 8bfbc55 · 4 minutes ago 1 Commit

 **README.md** first commit 4 minutes ago

 **README** 

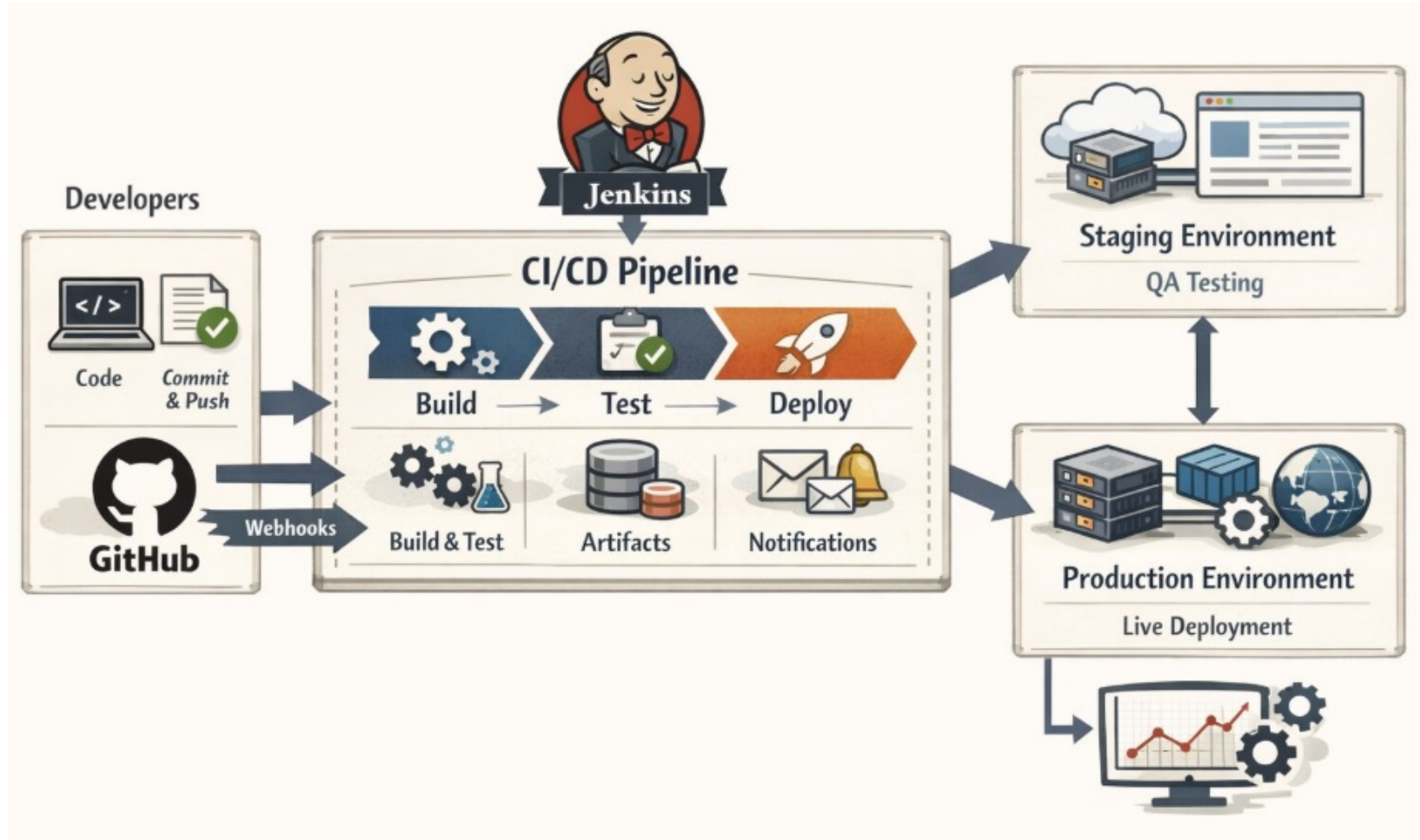
git-test



3. Jenkins

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Github + Jenkins CI/CD Pipeline



Jenkins 기본 환경 구성

\$ docker -v

Docker version 20.10.17, build 100c701

\$ docker-compose -v

Docker Compose version v2.10.2

docker-compose.yaml

version: "3.8"

컨테이너 내 볼륨 구성

services:

jenkins:

image: jenkins/jenkins:lts-jdk21

container_name: jenkins

restart: unless-stopped

user: root

ports:

- "8080:8080"

- "50000:50000"

environment:

- TZ=Asia/Seoul

volumes:

- jenkins_home:/var/jenkins_home

- /var/run/docker.sock:/var/run/docker.sock

volumes:

jenkins_home:

version: "3.8"

로컬 볼륨을 연결하는 볼륨 구성

services:

jenkins:

image: jenkins/jenkins:lts-jdk21

container_name: jenkins

restart: unless-stopped

user: root

ports:

- "8080:8080"

- "50000:50000"

environment:

- TZ=Asia/Seoul

volumes:

- ./jenkins_home:/var/jenkins_home

- /var/run/docker.sock:/var/run/docker.sock

\$ docker-compose up -d

```
▶ stevejung@Stevesui-iMac ~/KDT docker-compose up -d
WARN[0001] /Users/stevejung/KDT/docker-compose.yaml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 13/13
✓ jenkins Pulled 20.4s
  ✓ d0238388e632 Download complete 0.9s
  ✓ 9e6f96481dc6 Download complete 13.9s
  ✓ 8d5cd706e369 Download complete 0.9s
  ✓ e37c8a6a1d29 Download complete 0.6s
  ✓ e1d3077f0c0c Download complete 1.7s
  ✓ 768595d27f0b Download complete 11.5s
  ✓ 1944ded7dbca Download complete 0.9s
  ✓ b2b31b28ee3c Download complete 8.4s
  ✓ 37b0412849e4 Download complete 0.9s
  ✓ 2902ddfaf8af Download complete 0.5s
  ✓ 66714a60a07a Download complete 3.3s
  ✓ 0867b45f78b4 Download complete 0.9s
[+] Running 2/2
✓ Network kdt_default Created 0.1s
✓ Container jenkins Started 3.3s
```


Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

`/var/jenkins_home/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

```
$ docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword  
ee701c3a3eae4145a0a458da18596d14
```

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Getting Started

Getting Started

☒ Folders	☒ OWASP Markup Formatter	☒ Build Timeout	☒ Credentials Binding
☒ Timestampers	☒ Workspace Cleanup	☒ Ant	☒ Gradle
☒ Pipeline	☒ GitHub Branch Source	☒ Pipeline: GitHub Groovy Libraries	☒ Pipeline: Stage View
☒ Git	☒ SSH Build Agents	☒ Matrix Authorization Strategy	☒ PAM Authentication

 정상 설치가 안 되는 경우 교육장 Wi-Fi 가 아닌 개인 별 모바일 기기 핫스팟(테더링) 사용 권장.

Getting Started

Getting Started

☒ Folders	☒ OWASP Markup Formatter	☒ Build Timeout	☒ Credentials Binding	☒ SSH Credentials Binding
☒ Timestampers	☒ Workspace Cleanup	☒ Ant	☒ Gradle	☒ SSH API
☒ Pipeline	☒ GitHub Branch Source	☒ Pipeline: GitHub Groovy Libraries	☒ Pipeline Graph View	☒ Pipeline: API
☒ Git	☒ SSH Build Agents	☒ Matrix Authorization Strategy	☒ LDAP	☒ Timestampers
☒ Email Extension	☒ Mailer	☒ Dark Theme		☒ Caffeine API
				☒ Script Security
				☒ Pipeline: Supporting APIs
				☒ Plugin Utilities API
				☒ Font Awesome API
				☒ Bootstrap 5 API
				☒ JavaBeans Activation Framework (JAF) API
				☒ JAXB
				☒ SnakeYAML API
				☒ Jakarta Activation API
				☒ Jakarta XML Binding API
				☒ JSON API
				☒ Jackson 2 API
				☒ Jquery3 API
				☒ ECharts API
				☒ SnakeYAML Engine API
				☒ Jackson 3 API
				☒ Prime API
				☒ Display URL API
				☒ Check API
				☒ JUnit
				☒ Matrix Project
				☒ Resource Disposer
				☒ Workspace Cleanup
				☒ Ant
				☒ OkHttp
				☒ Apache HttpComponents Client 4.x API
				☒ Pipeline: Job
				☒ Gradle
				☒ Pipeline: Milestone Step
				☒ Durable Task
				☒ - required dependency

Create First Admin User

계정명

admin

암호

....

암호 확인

....

이름

이메일 주소

Instance Configuration

Jenkins URL:

http://localhost:8080/

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

 **Jenkins**

Q 검색 (⌘+K) ?

 1  Steve JUNG ▾

Dashboard >

+ 새로운 Item

 빌드 기록

 Jenkins 관리

 My Views

빌드 대기 목록 ▾

빌드 대기 항목이 없습니다.

빌드 실행 상태 0/2 ▾

Jenkins에 오신 것을 환영합니다.

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Set up a distributed build

Set up an agent 

Configure a cloud 

Learn more about distributed builds ?

```
pipeline {  
  agent any  
  stages {  
    stage('Build') {  
      steps {  
        //  
      }  
    }  
    stage('Test') {  
      steps {  
        //  
      }  
    }  
    stage('Deploy') {  
      steps {  
        //  
      }  
    }  
  }  
}
```

<https://www.jenkins.io/doc/book/pipeline/syntax/#declarative-pipeline>

```
node {  
  stage('Build') {  
    //  
  }  
  stage('Test') {  
    //  
  }  
  stage('Deploy') {  
    //  
  }  
}
```

<https://www.jenkins.io/doc/book/pipeline/syntax/#scripted-pipeline>

아르

OK

Dashboard > Declarative pipeline > Configuration

Configure



General



Advanced Project Options



Pipeline

General

설명

[Plain text] [미리보기](#)

☐

Do not allow concurrent builds

☐

Do not allow the pipeline to resume if it

Pipeline

Definition

Pipeline script



Script ?

1

[try sample Pipeline...](#)



Use Groovy Sandbox ?

[Pipeline Syntax](#)

Jenkins 기본 환경 구성

```

pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        echo '소스코드 가져오기'
        checkout scm
      }
    }
    stage('Build') {
      steps {
        echo '빌드 수행'
        sh 'echo "build start"'
      }
    }
    stage('Test') {
      steps {
        echo '테스트 수행'
        sh 'echo "test start"'
      }
    }
    stage('Deploy') {
      steps {
        echo '배포 수행'
        sh 'echo "deploy start"'
      }
    }
  }
}

```

Pipeline

Definition

Pipeline script

Script ?

```

6      echo 'building the application...'
7    }
8  }
9  stage('test') {
10    steps {
11      echo 'testing the application...'
12    }
13  }
14  stage('deploy') {
15    steps {
16      echo 'deploying the application...'
17    }
18  }
19 }
20 }
21

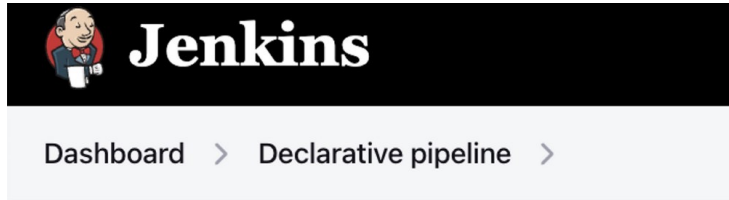
```

try sample Pipeline... ▾

☒ Use Groovy Sandbox ?
[Pipeline Syntax](#)

저장

Apply



Status

</> Changes

▶ 지금 빌드

⚙ 구성

🗑 Pipeline 삭제

🔍 Full Stage View

✎ Rename

❓ Pipeline Syntax

Dashboard > Declarative pipeline >

Status

</> Changes

▶ 지금 빌드

⚙ 구성

🗑 Pipeline 삭제

🔍 Full Stage View

✎ Rename

❓ Pipeline Syntax

Pipeline Declarative pipeline

Stage View

	build	test	deploy
Average stage times: (Average full run time: ~9s)	1s	1s	1s
#1 1월 22일 22:16 No Changes	1s	1s	1s

고정링크

- Last build, (#1), 2 min 43 sec 전
- Last stable build, (#1), 2 min 43 sec 전
- Last successful build, (#1), 2 min 43 sec 전
- Last completed build, (#1), 2 min 43 sec 전

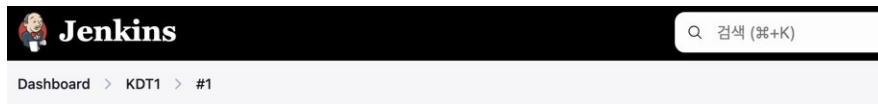
Build History 추이 ▼

🔍 Filter builds... /

#1
2023. 1. 22. 오후 10:16
[Atom feed \(전체\)](#) [Atom feed \(실패\)](#)

Jenkins 기본 환경 구성

왼쪽 탭에 위치한 **Console Output** 을 클릭해서 실행한 파이프라인의 전체 출력을 확인할 수 있다.



Status

✓ #1 (2024. 12. 6. 오전 10:39:26)

</> Changes

🔍 Console Output

✓ Edit Build Information

🗑 Delete build '#1'

🕒 Timings

🔗 Pipeline Overview

📄 Pipeline Console

🔄 Restart from Stage

🔄 Replay

☰ Pipeline Steps

📁 Workspaces



사용자 **Steve JUNG** 에 의해 시작됨



This run spent:

- 0.1 sec waiting;
- 7 sec build duration;
- 7.1 sec total from scheduled to completion.



변경사항 없음.

Status

✓ 콘솔 출력

</> Changes

🔍 Console Output

✓ Edit Build Information

🗑 Delete build '#1'

🕒 Timings

🔗 Pipeline Overview

📄 Pipeline Console

🔄 Restart from Stage

🔄 Replay

☰ Pipeline Steps

📁 Workspaces

```
Started by user Steve JUNG
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/KDT1
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build)
[Pipeline] echo
building the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (test)
[Pipeline] echo
testing the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (deploy)
[Pipeline] echo
deploying the application...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

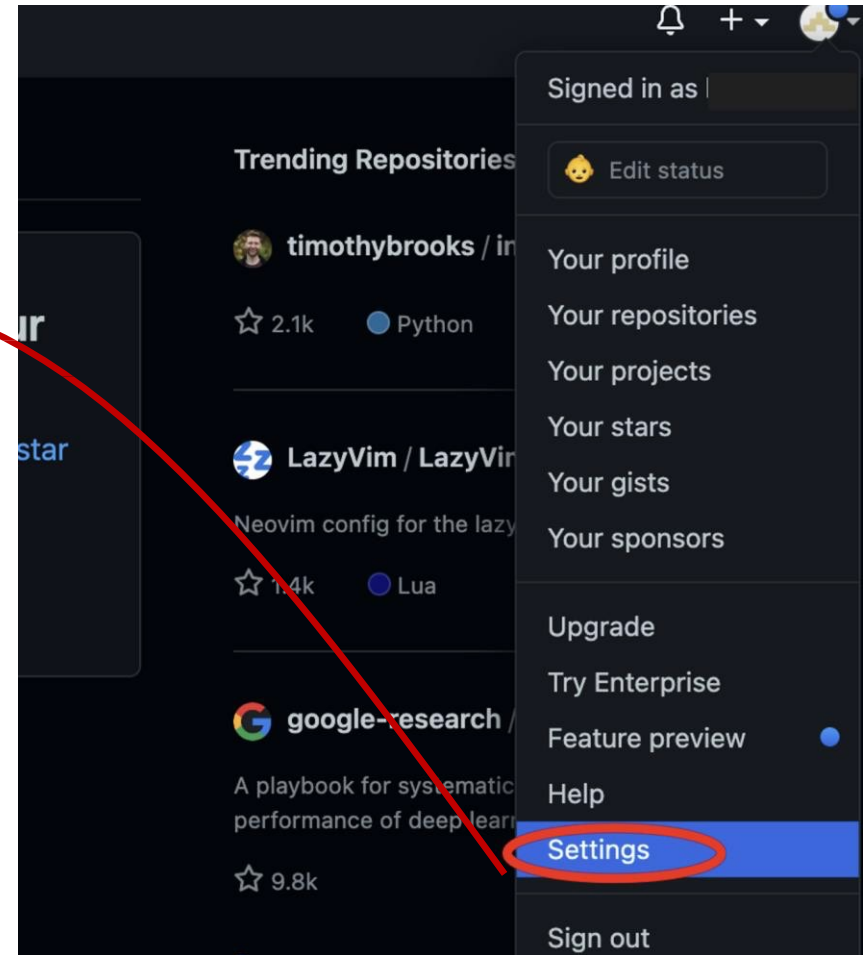
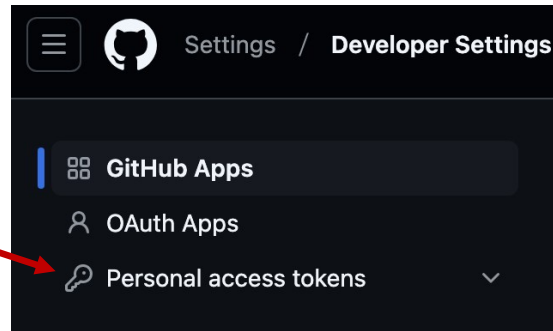
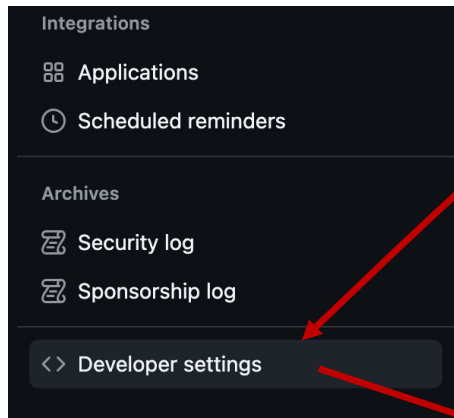
Github WebHooks 연동하여 Jenkins 빌드 자동화

GitHub Webhook을 사용하여 GitHub 리포지토리에 커밋이 발생할 때 Jenkins에서 자동으로 빌드되도록 설정

GitHub 웹사이트에서 프로필 아이콘을 클릭한다.

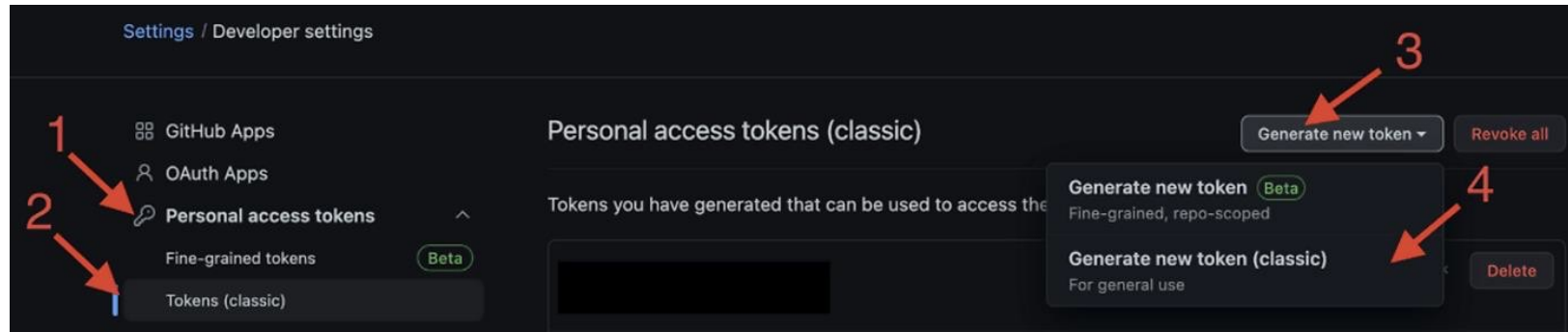
드롭다운 메뉴에서 "Settings"를 선택한다.

"Settings" 페이지에서 좌측 메뉴에서 "Developer settings"로 이동
Personal Access Tokens 메뉴로 이동한다.



Github WebHooks 연동하여 Jenkins 빌드 자동화

1. Personal access tokens 클릭
2. Tokens(classic) 클릭
3. Generate new token 클릭
4. Generate new token (classic) 클릭



Github WebHooks 연동하여 Jenkins 빌드 자동화

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note

jenkins

What's this token for?

Expiration *

30 days The token will expire on Thu, Feb 23 2023

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☐ write:packages	Upload packages to GitHub Package Registry
☐ read:packages	Download packages from GitHub Package Registry
☐ delete:packages	Delete packages from GitHub Package Registry
☐ admin:org	Full control of orgs and teams, read and write org projects
☐ write:org	Read and write org and team membership, read and write org projects
☐ read:org	Read org and team membership, read org projects
☐ manage_runners:org	Manage org runners and runner groups
☐ admin:public_key	Full control of user public keys
☐ write:public_key	Write user public keys
☐ read:public_key	Read user public keys
☒ admin:repo_hook	Full control of repository hooks
☒ write:repo_hook	Write repository hooks
☒ read:repo_hook	Read repository hooks

**repo 와 admin:repo_hook 를 선택해 주고
Generate token 버튼 클릭**

**토큰이 생성되면 보안 상의 이유로 토큰을 복사하여
안전한 장소에 보관하는 것이 중요**

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

✓ Ufk  Delete

Github WebHooks 연동하여 Jenkins 빌드 자동화

이제 생성한 GitHub Personal Access Token을 Jenkins에 등록하도록 하겠다.
Jenkins 관리 클릭 > Manage Credentials 클릭

The screenshot shows the Jenkins web interface. On the left sidebar, the 'Jenkins 관리' (Jenkins Management) option is highlighted with a red arrow and the number '1'. Below this, the 'Security' section is visible, and the 'Manage Credentials' option is highlighted with a red arrow and the number '2'. The main content area shows the 'Jenkins 관리' page with various configuration options like 'System Configuration', 'Global Tool Configuration', and 'Security'.

Jenkins 관리

Building on the built-in node can be a security issue. You should set up distributed builds. See [the documentation](#).

System Configuration

- 시스템 설정**
환경변수 및 경로 정보등을 설정합니다.
- Global Tool Configuration**
Configure tools, their locations and automatic installers.
- 플러그인 관리**
Jenkins의 기능을 확장하기 위한 플러그인을 추가, 제거, 사용, 미사용으로 설정할 수 있습니다.
- 노드 관리**
Add, remove, control and monitor the various nodes that Jenkins runs jobs on.

Security

- Configure Global Security**
Secure Jenkins; define who is allowed to access/use the system.
- Manage Credentials**
Configure credentials
- Configure Credential Providers**
Configure the credential providers and types
- Manage Users**
Create/delete/modify users that can log in to this Jenkins.

Github WebHooks 연동하여 Jenkins 빌드 자동화

Domains 밑에 있는 (global) 클릭

Dashboard > Jenkins 관리 > Credentials

Credentials

T P Store ↓ Domain

Stores scoped to Jenkins

P Store ↓ Domains

 System

(global)

아이콘: S M L

Github WebHooks 연동하여 Jenkins 빌드 자동화

Add Credentials 버튼 클릭

Jenkins 관리 > Credentials > System > Global credentials (unrestricted) >

Global credentials (unrestricted)

+ Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

ID

Name

Kind

Description

This credential domain is empty. How about [adding some credentials?](#)

아이콘:

S

M

L

Github WebHooks 연동하여 Jenkins 빌드 자동화

- Username은 Github ID 를 입력한다.
- Password는 위에서 생성한 Github Personal Access Token을 입력한다.
- ID필드에 식별 가능한 자격 증명 ID를 입력한다.
- 다 입력한 후 Create 버튼을 클릭한다.

New credentials

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

GITHUB ID

☐ Treat username as secret ?

Password ?

.....

ID ?

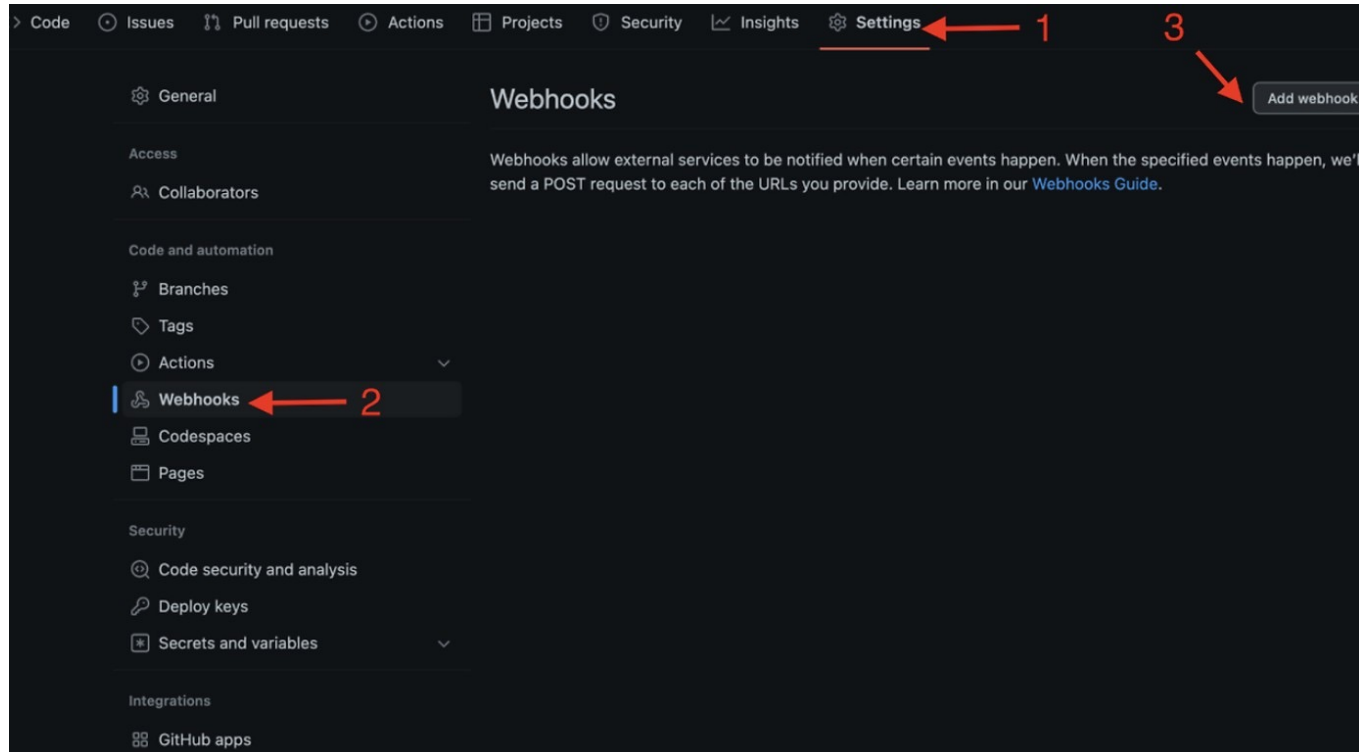
github_personal_access_token

Description ?

Create

Github WebHooks 연동하여 Jenkins 빌드 자동화

Jenkins와 연동할 Github Repository에 들어가서 Webhook를 생성한다.
Settings 클릭 > Webhooks 클릭 > Add webhook 버튼 클릭



Github WebHooks 연동하여 Jenkins 빌드 자동화

Payload URL에 젠킨스주소에 /github-webhook/ 를 추가해서 입력하면 된다.

Content type 은 application/json을 선택해 주고 Add webhook 버튼을 클릭하면 webhook이 생성된다.

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type

Secret

SSL verification

☐ By default, we verify SSL certificates when delivering payloads.

☒ Enable SSL verification ☐ Disable (not recommended)

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me everything.

☐ Let me select individual events.

☒ Active

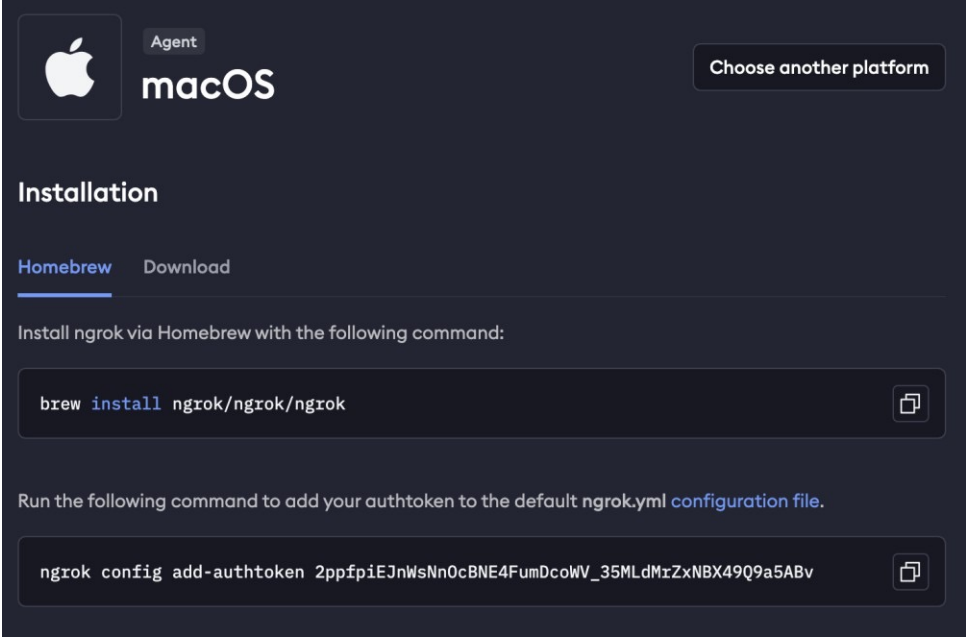
We will deliver event details when this hook is triggered.

Jenkins를 로컬에서 실행했기에 로컬환경을 인터넷으로 노출해줘야 한다.

ngrok를 이용하면 로컬에 구성한 환경을 외부에 공개할 수 있다.

Github WebHooks 연동하여 Jenkins 빌드 자동화

ngrok 설치



The image shows the ngrok website's macOS installation page. At the top, there's a dark header with the Apple logo, the word 'Agent', 'macOS', and a button that says 'Choose another platform'. Below this, the 'Installation' section is active, with 'Homebrew' and 'Download' tabs. The 'Homebrew' tab is selected, showing instructions to install ngrok via Homebrew with the command: `brew install ngrok/ngrok/ngrok`. Below this, there's a note to run a command to add an auth token to the default ngrok.yml configuration file, with the command: `ngrok config add-authtoken 2ppfpiEJnWsNn0cBNE4FumDcowV_35MLdMrZxNBX49Q9a5ABv`. Both command boxes have a copy icon on the right.

Agent macOS Choose another platform

Installation

Homebrew Download

Install ngrok via Homebrew with the following command:

```
brew install ngrok/ngrok/ngrok
```

Run the following command to add your authtoken to the default ngrok.yml configuration file.

```
ngrok config add-authtoken 2ppfpiEJnWsNn0cBNE4FumDcowV_35MLdMrZxNBX49Q9a5ABv
```


Github WebHooks 연동하여 Jenkins 빌드 자동화

ngrok

👋 Goodbye tunnels, hello Agent Endpoints: <https://ngrok.com/r/aep>

Session Status

online

Account

stevejung37@gmail.com (Plan: Free)

Version

3.18.4

Region

Japan (jp)

Latency

35ms

Web Interface

<http://127.0.0.1:4040>

Forwarding

<https://6b36-211-45-60-5.ngrok-free.app> -> <http://localhost:8080>

Connections

ttl	opn	rt1	rt5	p50	p90
1	0	0.00	0.00	30.96	30.96

HTTP Requests

16:04:36.397 KST POST /github-webhook/

200 OK

Github WebHooks 연동하여 Jenkins 빌드 자동화

하기 Pipeline script를 Jenkinsfile로 Github repository에 추가

```
pipeline {
    agent any
    stages {
        stage('build') {
            steps {
                echo 'building the application...'
            }
        }
        stage('test') {
            steps {
                echo 'testing the application...'
            }
        }
        stage('deploy') {
            steps {
                echo 'deploying the application...'
            }
        }
    }
}
```

Github WebHooks 연동하여 Jenkins 빌드 자동화

Jenkins pipeline 생성

Enter an item name

github-webhook

» Required field

Freestyle project
이것은 Jenkins의 주요 기능입니다. Jenkins은 어느 빌드 시스템과 어떤 SCM(형상관리)으로 묶인 당신의 프로젝트를 빌드할 것이고, 소프트웨어 빌드보다 다른 어떤 것에 자주 사용될 수 있습니다.

Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Multi-configuration project
다양한 환경에서의 테스트, 플래폼 특성 빌드, 기타 동등 처럼 다수의 서로다른 환경설정이 필요한 프로젝트에 적합함.

Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder
Creates a set of multibranch project subfolders by scanning for repositories.

If you want to create a new item from other existing, you can use this option:

Copy from

Type to autocomplete

OK

General

Enabled

설명

[Plain text] [미리보기](#)

☐ Do not allow concurrent builds

☐ Do not allow the pipeline to resume if the controller restarts

☒ GitHub project

Project url ?

https://github.com/

Enter the URL for the GitHub hosted project (without the tree/master or tree/branch part).

For example: https://github.com/rails/rails for the Rails project.

(from [GitHub plugin](#))

고급...

☐ Pipeline speed/durability override ?

☐ Preserve stashes from completed builds ?

☐ Throttle builds ?

☐ 오래된 빌드 삭제 ?

☐ 이 빌드는 매개변수가 있습니다 ?

Build Triggers

☐ Build after other projects are built ?

☐ Build periodically ?

☒ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

☐ Quiet period ?

☐ 빌드를 원격으로 유발 (예: 스크립트 사용) ?

Github WebHooks 연동하여 Jenkins 빌드 자동화

Pipeline

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/cc202501/git-test.git

Credentials ?

SteveJUNG/***** (KDT)

+ Add

고급 ▾

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

Add Branch

Repository browser ?

(자동)

Additional Behaviours

Add ▾

Script Path ?

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

저장

Apply

Github WebHooks 연동하여 Jenkins 빌드 자동화

Dashboard >

+ 새로운 Item

📁 빌드 기록

😊 프로젝트 연관 관계

🖨 파일 핑거프린트 확인

⚙ Jenkins 관리

S	W	Name ↓	최근 상태
...	☀	github-webhook	—

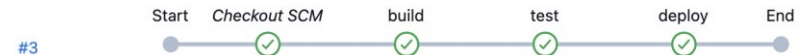
```

stevejung@Stevesui-iMac ~/KDT/Sample/git-test ⌕ main ➤ vi README.md
stevejung@Stevesui-iMac ~/KDT/Sample/git-test ⌕ main ± 
stevejung@Stevesui-iMac ~/KDT/Sample/git-test ⌕ main ± ➤ git add .
stevejung@Stevesui-iMac ~/KDT/Sample/git-test ⌕ main + ➤ git commit -m "webhook testing"
[main acfeb2] webhook testing
1 file changed, 1 insertion(+)
stevejung@Stevesui-iMac ~/KDT/Sample/git-test ⌕ r main ➤ git push
오브젝트 나열하는 중: 5, 완료.
오브젝트 개수 세는 중: 100% (5/5), 완료.
Delta compression using up to 4 threads
오브젝트 압축하는 중: 100% (3/3), 완료.
오브젝트 쓰는 중: 100% (3/3), 325 bytes | 325.00 KiB/s, 완료.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/cc202501/git-test.git
fd8a7e7..acfeb2 main -> main
  
```

Dashboard > github-webhook > Stages

Build github-webhook

id ↓ pipeline



4. Tekton

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Tekton 적용 흐름

✅ Tekton 적용 흐름

1. Kubernetes 클러스터에 **Tekton Pipelines**를 설치
2. 빌드/테스트/배포 단위를 **Task**로 정의
3. 여러 **Task**를 묶어서 **Pipeline**을 정의
4. 실제 실행은 **TaskRun** 또는 **PipelineRun**으로..
5. GitHub Webhook 같은 외부 이벤트로 자동 실행하려면 **Triggers**를 설치하고
EventListener, **TriggerBinding**, **TriggerTemplate**를 정의
6. 실행 상태는 **kubectl**, **tkn**, **Dashboard**로 확인

Tekton?

Tekton은 **Kubernetes 네이티브 CI/CD 프레임워크**로, 파이프라인을 Kubernetes 리소스(Task, Pipeline 등)로 정의하고 실행한다.

✓ 주요 특징

- Kubernetes 네이티브: 모든 구성 요소가 CRD(Custom Resource Definition)로 정의
- 컨테이너 기반: 각 Step이 독립 컨테이너로 실행
- 선언적: YAML로 파이프라인 정의, 버전 관리 용이
- 재사용성: Task를 조합하여 Pipeline 구성

✓ Tekton 프로젝트 구성

프로젝트	설명
Pipelines	Task, Pipeline, PipelineRun 등 핵심 리소스
Triggers	이벤트 기반 파이프라인 실행 (Webhook 등)
Dashboard	웹 UI로 파이프라인 모니터링
CLI	명령어 도구

! Tekton은 Kubernetes 위에서 동작한다. Docker만으로는 실행 불가하며, Docker Desktop Kubernetes, Kind, Minikube 등으로 로컬 K8s 클러스터를 구성한 후 Tekton을 설치한다.

Tekton 설치

Tekton은 Kubernetes 환경에서 CI/CD 파이프라인을 구축하고 실행하기 위한 오픈 소스 프레임워크로 컨테이너 기반의 파이프라인을 정의하고 실행하는 데 최적화된 솔루션으로 최근 Kubernetes에서 폭넓게 사용

✓ Tekton Pipelines

1 최신 릴리스 설치

kubectl apply -f <https://storage.googleapis.com/tekton-releases/pipeline/latest/release.yaml>

2 특정 버전 설치 (예: v0.52.0)

kubectl apply -f <https://storage.googleapis.com/tekton-releases/pipeline/previous/v0.52.0/release.yaml>

3 설치 모니터링

kubectl get pods -n tekton-pipelines -w

tekton-pipelines-controller와 tekton-pipelines-webhook 이 1/1 READY가 되면 완료.

Tekton 설치

✅ Tekton Triggers (선택, Webhook 등 이벤트 기반 실행)

Triggers 설치 (Pipelines 설치 후)

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/triggers/latest/release.yaml
```

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/triggers/latest/interceptors.yaml
```

✅ Tekton Dashboard (선택, 웹 UI)

읽기 전용 모드 (기본)

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/dashboard/latest/release.yaml
```

읽기/쓰기 모드 (파이프라인 실행 등)

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/dashboard/latest/release-full.yaml
```

Tekton 설치

✅ Dashboard 접속

포트 포워딩

kubectl port-forward -n tekton-pipelines svc/tekton-dashboard 9097:9097

브라우저에서 <http://localhost:9097> 접속

✅ Tekton Operator (프로덕션 환경 내 권장)

Operator로 Pipelines, Triggers, Chains, Dashboard 를 통합 관리

kubectl apply -f <https://storage.googleapis.com/tekton-releases/operator/latest/release.yaml>

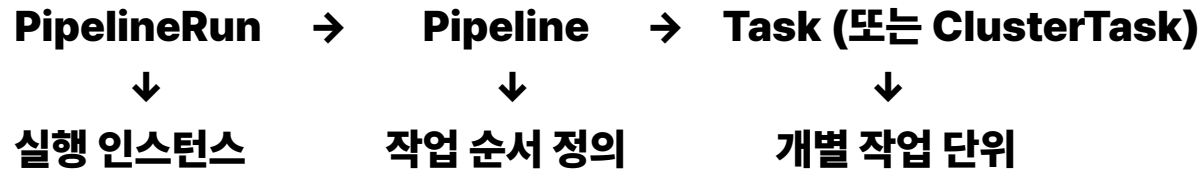
✅ Tekton CLI (tkn) 설치

brew install tektoncd-cli

tkn version

tkn help

리소스 계층 구조



구분	Task	ClusterTask
Scope	Namespace	클러스터 전체
사용 시점	프로젝트 별 작업	공통 작업

✅ Workspace

Task/Pipeline 간 파일을 공유하는 볼륨. `emptyDir`, `PersistentVolumeClaim`, `ConfigMap` 등 사용

✅ Params vs Results

- Params: Task/Pipeline 실행 시 전달하는 입력값
- Results: Task가 출력하는 값, 후속 Task에서 `$(tasks.TASK_NAME.results.RESULT_NAME)`로 참조

Task 작성 및 실행

기본 Task 예시

hello-task.yaml

apiVersion: tekton.dev/v1

kind: Task

metadata:

name: hello-task

spec:

steps:

- name: echo

image: alpine:3.20

script: |

#!/bin/sh

echo "Hello Tekton"

kubectl apply -f hello-task.yaml

apiVersion: tekton.dev/v1

kind: TaskRun

metadata:

name: hello-task-run

spec:

taskRef:

name: hello-task

kubectl apply -f hello-task-run.yaml

kubectl get taskrun

kubectl describe taskrun hello-task-run

Task 생성 (실습)

build-task

```
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: build-task
spec:
  params:
    - name: app-name
      type: string
  steps:
    - name: build
      image: alpine:3.20
      script: |
        #!/bin/sh
        echo "Building $(params.app-name)"
```

test-task

```
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: test-task
spec:
  params:
    - name: app-name
      type: string
  steps:
    - name: test
      image: alpine:3.20
      script: |
        #!/bin/sh
        echo "Testing $(params.app-name)"
```

Pipeline 및 PipelineRun (실습)

```

apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: sample-pipeline
spec:
  params:
    - name: app-name
      type: string
  tasks:
    - name: build
      taskRef:
        name: build-task
      params:
        - name: app-name
          value: $(params.app-name)
    - name: test
      runAfter:
        - build
      taskRef:
        name: test-task
      params:
        - name: app-name
          value: $(params.app-name)

```

```

apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  name: sample-pipeline-run
spec:
  pipelineRef:
    name: sample-pipeline
  params:
    - name: app-name
      value: my-app

```

```

kubectl apply -f build-task.yaml
kubectl apply -f test-task.yaml
kubectl apply -f pipeline.yaml
kubectl apply -f pipelinerun.yaml

```

```

kubectl get pipelinerun
kubectl get taskrun
kubectl describe pipelinerun sample-pipeline-run

```

✓ 로그 확인

```
tkn pipelinerun logs sample-pipeline-run -f
```

실전 활용 예시

git-clone ClusterTask 설치 (Tekton Catalog)

kubectl apply -f <https://raw.githubusercontent.com/tektoncd/catalog/main/task/git-clone/0.11/git-clone.yaml>

✓ clone-pipeline.yaml (Git 저장소 클론)

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: clone-and-build
spec:
  params:
    - name: repo-url
      type: string
    - name: revision
      type: string
      default: main
  workspaces:
    - name: shared-workspace
  tasks:
    - name: fetch-source
      taskRef:
        kind: ClusterTask
        name: git-clone
      params:
        - name: url
          value: $(params.repo-url)
        - name: revision
          value: $(params.revision)
      workspaces:
        - name: output
          workspace: shared-workspace
```

Kaniko는 Dockerfile 없이 컨테이너 내부에서 이미지를 빌드한다.

✓ build-push-pipeline.yaml (빌드 & 푸시)

```
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: build-and-push
spec:
  params:
    - name: image-url
      type: string
    - name: dockerfile-path
      type: string
      default: "./Dockerfile"
  workspaces:
    - name: source
  tasks:
    - name: build-push
      taskRef:
        kind: ClusterTask
        name: kaniko
      params:
        - name: IMAGE
          value: $(params.image-url)
        - name: DOCKERFILE
          value: $(params.dockerfile-path)
      workspaces:
        - name: source
          workspace: source
```


CI-Pipeline

ci-pipeline.yaml

apiVersion: tekton.dev/v1

kind: Pipeline

metadata:

name: ci-pipeline

spec:

params:

- **name: repo-url**
type: string
- **name: image-url**
type: string

workspaces:

- **name: shared-data**

tasks:

- **name: clone**

taskRef:

kind: ClusterTask

name: git-clone

params:

- **name: url**
value: \$(params.repo-url)

workspaces:

- **name: output**
workspace: shared-data

- **name: test**

runAfter: [clone]

taskRef:

name: run-tests

workspaces:

- **name: source**
workspace: shared-data

- **name: build**

runAfter: [test]

taskRef:

kind: ClusterTask

name: kaniko

params:

- **name: IMAGE**
value: \$(params.image-url)

workspaces:

- **name: source**
workspace: shared-data

Workspace 사용 예시

✓ PipelineRun with PVC

apiVersion: tekton.dev/v1

kind: PipelineRun

metadata:

name: ci-pipeline-run

generateName: ci-run-

spec:

pipelineRef:

name: ci-pipeline

params:

- name: repo-url

value: "https://github.com/username/repo.git"

- name: image-url

value: "myregistry.io/myimage:latest"

workspaces:

- name: shared-data

volumeClaimTemplate:

spec:

accessModes:

- ReadWriteOnce

resources:

requests:

storage: 1Gi

5. Github Action

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Github Action?

GitHub Actions는 GitHub에서 제공하는 CI/CD 플랫폼으로, 빌드·테스트·배포를 자동화할 수 있다.

쉬운 관점에서 “이벤트가 발생하면 정해진 서버(runner)에서 정해진 절차(workflow/job/step)를 자동 실행”하는 구조이다.

✓ 주요 특징

- GitHub 통합: 저장소와 완벽 통합, 별도 서버 불필요
- YAML 기반: ``.github/workflows/``에 워크플로우 정의
- 무료 티어: 퍼블릭 저장소 무제한, 프라이빗 저장소 월 2,000분
- 마켓플레이스: 수천 개의 재사용 가능한 Action

✓ 구성 요소

구성요소	설명
Workflow	자동화된 프로세스 (YAML 파일) , ci.yaml, deploy.yaml 등
Event	워크플로우를 트리거하는 활동, push, pull_request, workflow_dispatch, schedule 등
Job	같은 Runner에서 실행되는 Step 집합, 워크플로 안의 작업 단위 각 job은 별도 runner에서 실행될 수 있고, 기본적으로 서로 병렬 실행된다. needs를 사용하면 선행 job이 끝난 뒤 실행되게 만들 수 있다.
Step	개별 작업 (스크립트 또는 Action), Job 내부의 세부 실행 단계
Action	재사용 가능한 작업 단위, actions/checkout, actions/setup-node 등
Runner	워크플로우를 실행하는 서버, 실제로 Job이 실행되는 환경

Github Action 핵심 개념

✅ Workflow

- `.github/workflows/` 디렉토리에 YAML 파일로 정의
- 파일 확장자: `.yml` 또는 `.yaml`
- 저장소당 여러 워크플로우 가능

✅ Runner

- GitHub-hosted: Ubuntu, Windows, macOS (GitHub 제공)
- Self-hosted: 자체 인프라에서 실행

✅ 실행 흐름

Event 발생 → Workflow 실행 → Job(s) 실행 → Step(s) 실행

워크플로우 기본 구조

✓ 최소 예시

```
yaml 타입
# .github/workflows/hello.yml
name: Hello Workflow
on: push
jobs:
  say-hello:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Hello, GitHub Actions!"
```

name: hello-actions

on:
push:
branches: [main]

jobs:
hello:
runs-on: ubuntu-latest
steps:
- name: Print message
run: echo "Hello GitHub Actions"

워크플로 이름: hello-actions
main 브랜치 push 시 실행
ubuntu-latest runner에서 실행
echo 명령 수행

워크플로우 전체 구조

```

name: Workflow 이름
run-name: ${{ github.actor }}의 실행 # 실행별 표시 이름
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
env:
  NODE_VERSION: '20'
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4
      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
      - name: Install and Test
        run: |
          npm ci
          npm test

```

```

name: ci-cd

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

  build:
    runs-on: ubuntu-latest
    needs: test
    steps:
      - uses: actions/checkout@v4

  deploy:
    runs-on: ubuntu-latest
    needs: build
    if: ${{ github.ref == 'refs/heads/main' }}
    steps:
      - run: echo "deploy"

```

이벤트	설명
push	브랜치에 푸시, 커밋이 브랜치나 태그로 푸시될 때 실행
pull_request	PR 생성/업데이트, PR이 열리거나, 업데이트되거나, 다시 열릴 때 실행
workflow_dispatch	수동 실행 , GitHub UI에서 사람이 직접 버튼으로 실행하는 수동 이벤트이다. 배포, 점검, 재 실행성 작업에 자주 사용
repository_dispatch	GitHub 외부 시스템이 GitHub API를 호출해서 워크플로를 시작할 때 쓰는 이벤트
schedule	cron 스케줄, GitHub는 여러 schedule을 한 workflow에 둘 수 있고, 높은 부하 시간대에는 지연될 수 있다고 안내함.
release	릴리스 생성/발행 시 실행, 버전 배포 자동화에 적합, 릴리스 생성 시 워크플로를 트리거할 수 있음.
issues	이슈 생성, 수정, 라벨 추가 등 이슈 관련 활동에 반응할 수 있음

이벤트 필터링

이벤트는 단순히 선언만 하는 것이 아니라 브랜치, 태그, activity type으로 더 좁힐 수 있다.

이벤트 여러 개를 한 워크플로에 같이 쓰기

```
on:
  push:
    branches:
      - main
      - develop

on:
  push:
    tags:
      - "v*"

on:
  pull_request:
    types: [opened, synchronize, reopened]
```

```
name: ci-cd

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  show-event:
    runs-on: ubuntu-latest
    steps:
      - run: echo "event = ${github.event_name}"
```

이벤트

✓ push

```
on:
  push:
    branches: [main, develop]
    paths:
      - 'src/'
      - 'package.json'
```

✓ pull_request

```
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]
```

✓ workflow_dispatch (수동 실행)

```
on:
  workflow_dispatch:
    inputs:
    environment:
      description: '배포 환경'
      required: true
      default: 'staging'
      type: choice
    options:
      - staging
      - production
```

✓ schedule (cron)

```
on:
  schedule:
    - cron: '0 0 * * *' # 매일 자정 (UTC)
```

✓ 여러 이벤트 조합

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  workflow_dispatch:
```

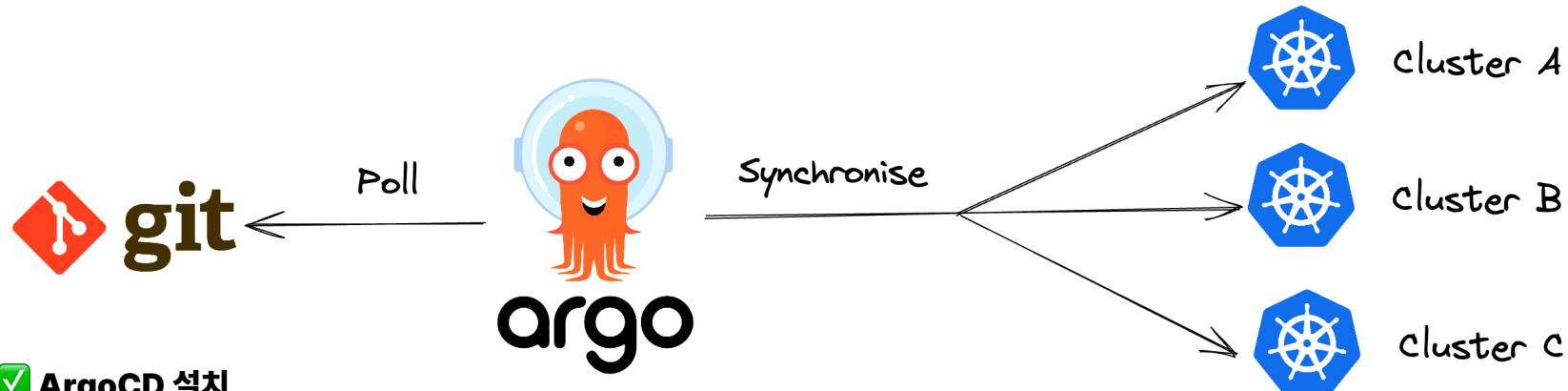
Vue.js + Springboot + Harbor Registry + K8s on Docker or EKS

6. GitOps 와 ArgoCD

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

ArgoCD ?

Kubernetes 환경에서 GitOps 방식으로 애플리케이션을 배포하고 관리하는 CD(Continuous Delivery) 도구이다.
 Git 저장소를 Single Source of Truth (SSOT) 로 사용하여 Kubernetes 클러스터의 상태를 자동으로 동기화한다.
 쉽게 말하면 Git에 선언된 Kubernetes 매니페스트 상태 = 실제 클러스터 상태가 되도록 자동으로 맞춰주는 시스템



✓ ArgoCD 설치

```
> k create ns argocd
```

```
> k apply -n argocd --server-side --force-conflicts \
```

```
-f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

✓ argocd-server 서비스 port-forward

```
> kubectl port-forward svc/argocd-server -n argocd 8080:443
```

✓ ArgoCD CLI 설치

```
> brew install argocd
```

✓ 초기 PW 확인 및 PW 변경

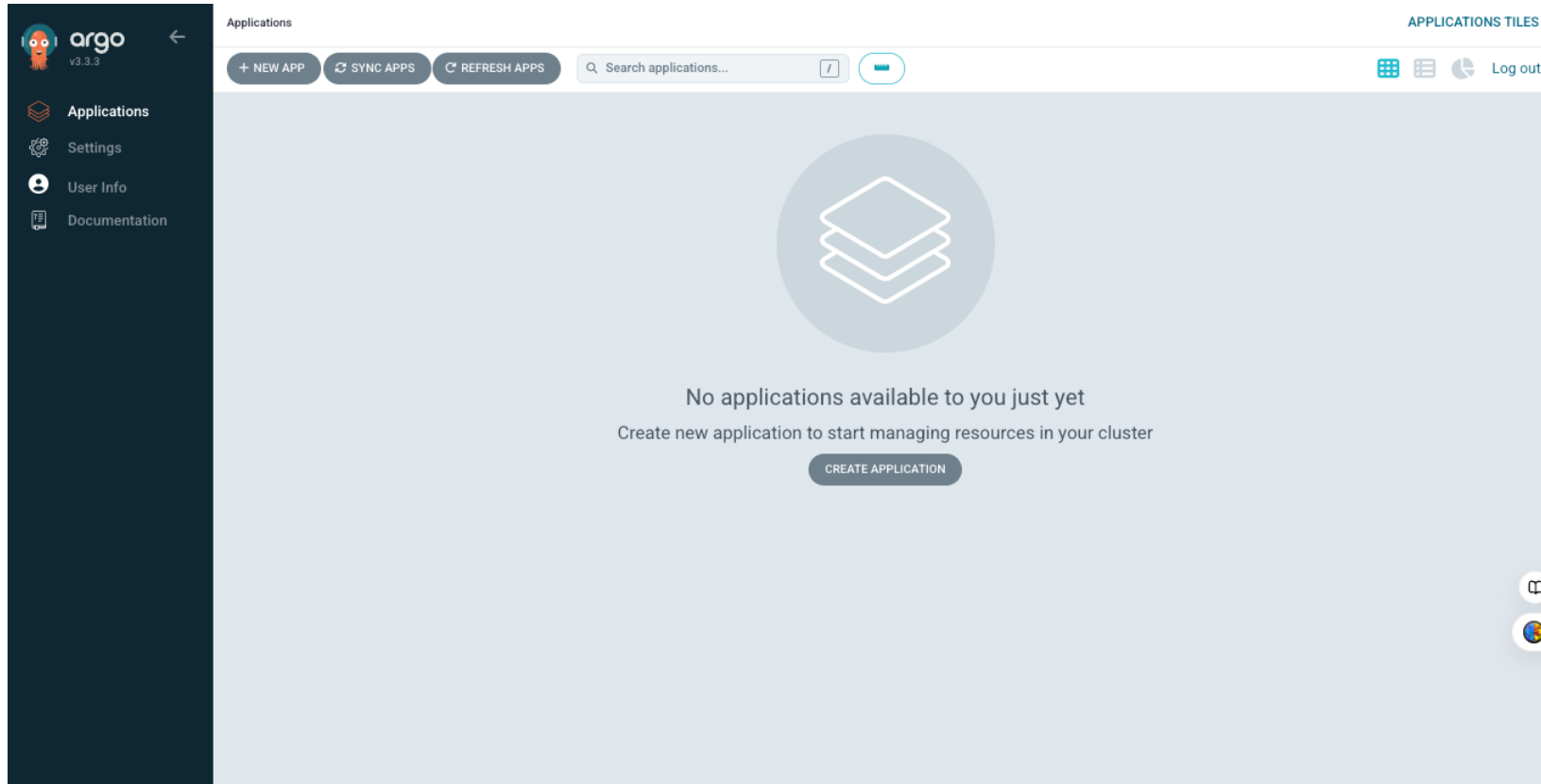
```
argocd admin initial-password -n argocd
```

```
argocd login localhost:8080
```

```
argocd account update-password
```

브라우저에서 ArgoCD 접속

localhost:8080



<https://github.com/skala202501/skala3>

The screenshot shows the GitHub repository page for `skala202501 / skala3`. The repository is viewed at the `main` branch. The left sidebar shows the file explorer with the `sample-app` directory expanded, listing `deployment.yaml`, `kustomization.yaml`, `namespace.yaml`, and `service.yaml`. The main content area shows the commit history for the `sample-app` directory, with a table listing the files and their last commit messages.

Repository: skala202501 / skala3

Navigation: <> Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

Files

main

Go to file

sample-app

- deployment.yaml
- kustomization.yaml
- namespace.yaml
- service.yaml

skala3 / sample-app

skala202501 sample-add commit

Name	Last commit message
..	
deployment.yaml	sample-add commit
kustomization.yaml	sample-add commit
namespace.yaml	sample-add commit
service.yaml	sample-add commit

<https://github.com/skala202501/skala3>

scala202501 / skala3

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Files

main

Go to file

argocd

application-sample-app.yaml

application-sample-app2.yaml

sample-app

deployment.yaml

kustomization.yaml

namespace.yaml

service.yaml

scala3 / sample-app /

scala202501 sample-app update3

Name	Last commit message
..	
deployment.yaml	sample-app update3
kustomization.yaml	sample-add commit
namespace.yaml	sample-add commit
service.yaml	sample-app update3

The screenshot displays the ArgoCD v3.3.3 web interface. On the left is a dark sidebar with navigation links: Applications, Settings, User Info, and Documentation. Below these are filters for 'Favorites Only' and 'SYNC STATUS' (Unknown: 0, Synced: 2, OutOfSync: 0). The 'HEALTH STATUS' section shows: Progressing (0), Suspended (0), Healthy (2), Degraded (0), Missing (0), and Unknown (0).

The main content area, titled 'Applications', features buttons for '+ NEW APP', 'SYNC APPS', and 'REFRESH APPS', along with a search bar. Two application cards are shown:

- sample-app**: Project: default, Labels: (empty), Status: Healthy Synced, Repository: https://github.com/argoproj/argocd-example-apps.git, Target Revision: HEAD, Path: kustomize-guestbook, Destination: in-cluster, Namespace: sample-app, Created At: 03/10/2026 15:31:32 (an hour ago), Last Sync: 03/10/2026 16:19:11 (9 minutes ago). Buttons: SYNC, REFRESH, DELETE.
- sample-app2**: Project: default, Labels: (empty), Status: Healthy Synced, Repository: https://github.com/skala202501/skala3.git, Target Revision: HEAD, Path: sample-app, Destination: in-cluster, Namespace: sample-app, Created At: 03/10/2026 16:19:42 (9 minutes ago), Last Sync: 03/10/2026 16:26:59 (a minute ago). Buttons: SYNC, REFRESH, DELETE.



수고하셨습니다.

본 문서는 SK(주) AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.